

PA2 实验报告

理解YEMU的执行过程

画出YEMU上执行的加法程序的状态机

YEMU - 用C程序实现ZSA状态机

$$S = \{cpc, R, M\} = \{PC, rd, r1, M\}$$
$$S_0 = (0, 0, 0, M) \rightarrow S_1 = (1, 6, 0, M) \rightarrow S_2 = (2, 6, 6, M)$$
$$\rightarrow S_3 = (3, 5, 6, M) \rightarrow S_4 = (4, 11, 6, M) \rightarrow S_5 = (5, 11, 6, M)$$

$\rightarrow \text{halt}$

来自华为笔记

通过RTFSC理解YEMU如何执行一条指令

YEMU执行指令的过程就是取指令，分析指令（操作码译码，操作数译码）和执行指令的过程

RTFSC请整理一条指令在NEMU中的执行过程

NEMU执行指令的过程为：

1. 在sdb键入si, 调用cpu_exec函数并传入参数1 \
2. 在cpu_exec函数中，先检测NEMU的state若为running则顺利调用execute函数并传入参数1 \
3. 在execute函数中则会调用一次exec_once函数并传入带有各种PC和log的字段的对象s，执行记录trace和diffest工具，再更新外设状态 \
4. 在exec_once函数中，先将pc和静态pc(用于后续更新静态pc)记录进s中并传入isa_exec_once函数，再更新pc为动态next_pc \
5. 在isa_exec_once函数中通过inst_fetch函数取出指令(取值就是从vaddr_ifetch->paddr_read->pmem_read->host_read)并更新静态PC \
6. 再执行decode_exec通过更新动态PC返回下一条指令的PC，并分析指令和执行指令

程序如何运行 理解打字小游戏如何运行

1. NEMU作为硬件模拟器用于模拟键盘（产生按键事件）和显示设备（更新屏幕显示），并将按键信息存储到内存映射的I/O区域(区域定义在./abstract-machine/am/src/platform/nemu/include/nemu.h中),具体啊的说,当在键盘上按下'A'时, NEMU中会在device_update中的事物变量中将event.type设置为KEYUP,并将对应按键的scancode存入k中,当松开按键时会通过send_key将k存入键盘队列（键盘外设的内存中）\
2. ISA层面上, 将io_read(AM_INPUT_KEYBRD)编译成对应ARCH的指令通过访存指令将scancode存入键盘外设寄存器(KBD_ADDR)中\
3. 在AM中的ioe/input.c中的__am_input_keybrd函数中从键盘外设寄存器所映射的内存中取出scancode\
4. 在打字小游戏程序进行完上述动作后, 就可以通过check_hit检测这个按键是否命中了\
5. 对应VGA图形外设, 则会先在am中的\$ARCH/ioe/gpu.c将像素数据写入显存用于渲染图形, 再在NEMU中更新显示窗口

编译与链接

1. 在nemu/include/cpu/ifetch.h中, 你会看到由static inline开头定义的inst_fetch()函数. 分别尝试去掉static, 去掉inline或去掉两者, 然后重新进行编译, 你可能会看到发生错误. 请分别解释为什么这些错误会发生/不发生? 你有办法证明你的想法吗?

1. 去掉static(保留inline)能够编译成功:\

原因: inline会告诉编译器将函数调用替换为函数体代码就算没有static编译器也会在每个调用处内联展开函数, 且由于是内联函数所以不会产生函数符号就不会有重复定义错误

2. 去掉inline(保留static)也能编译成功:\

原因: static会限制inst_fetch的作用域为当前文件下, 包含这个头文件的.c文件都会有自己独立的inst_fetch函数副本所以不会产生符号冲突

3. 同时去掉inline和static就会重复定义:\

原因: 没有static函数将具有外部链接的能力, 会出现在全局符号表中, 没有inline编译器则会为每个包含这个头文件的.c文件生成函数符号, 当进入链接阶段后链接器就会发现多个同名的全局符号就会报错

2. 在nemu/include/common.h中添加一行volatile static int dummy; 然后重新编译NEMU. 请问重新编译后的NEMU含有多少个dummy变量的实体? 你是如何得到这个结果的?

加上volatile static int dummy后可以看到一共有35个dummy实体, 可以使用nm查看符号表(使用nm /riscv32-nemu-interpreter | grep dummy | wc -l)或者反汇编后grep dummy, 还可以查看目标文件(.o)中的dummy和readelf查看符号表

3.添加上题中的代码后, 再在nemu/include/debug.h中添加一行 `volatile static int dummy;` 然后重新编译NEMU. 请问此时的NEMU含有多少个dummy变量的实体? 与上题中dummy变量实体数目进行比较, 并解释本题的结果.

在debug.h中添加后, 一共还是35个dummy实体(没有变化)\

原因: common.h中include的了debug.h所以两者在同一个编译单元中, 编译器将其视为了重复定义, 最终只生成了一个实体所以总数没有增加。

4.修改添加的代码, 为两处dummy变量进行初始化:`volatile static int dummy = 0;` 然后重新编译NEMU. 你发现了什么问题? 为什么之前没有出现这样的问题? (回答完本题后可以删除添加的代码.)

没有初始化dummy时, dummy被视为是一个弱符号, 链接器允许多个相同的弱符号存在并会自动合并它们, 且dummy是static所以每个编译单元都有自己的局部符号, 初始化后dummy成为了强符号当链接器发现多个相同名称的强符号时就会有重复定义的错误, 弱符号会在符号表中标记为BSS段而, 强符号则会标记为DATA段

了解Makefile

请描述你在am-kernels/kernels/hello/目录下敲入make ARCH=\$ISA-nemu 后, make程序如何组织.c和.h文件, 最终生成可执行文件am-kernels/kernels/hello/build/hello-\$ISA-nemu.elf.

首先Makefile会从hello/Makefile到abstract-machine/Makefile再到abstract-machine/scripts/\$ISA-nemu.mk中同时设置好变量NAME, SRCS, ARCH, ISA\

在编译阶段会通过(`@$(CC) -std=gnu11 $(CFLAGS) -c -o $@ $(realpath $<)`)将hello.c编译成目标文件hello.o\

在构建AM库通过生成\$(1)-\$(ARCH).a实际上就是am-riscv32-nemu.a和klib-riscv32-nemu.a, am中规定了对应架构的程序入口地址start.S\

再通过链接规则: Rule (link): objects (`*.o`) and libraries (`*.a`) -> `IMAGE.elf`, the final ELF binary to be packed into image (ld)链接生成elf文件和bin文件,并在NEMU中运行这个程序